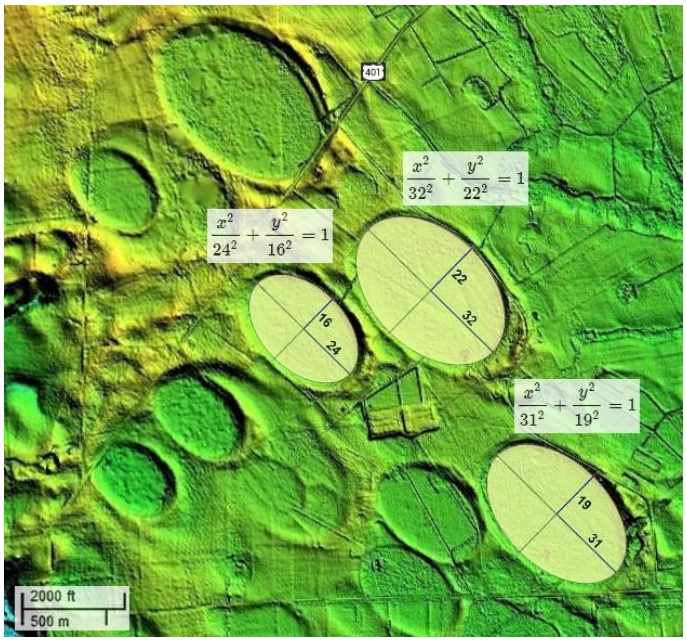
Fitting ellipses to Carolina Bays by the Least Squares Method

Antonio Zamora, May 10, 2026

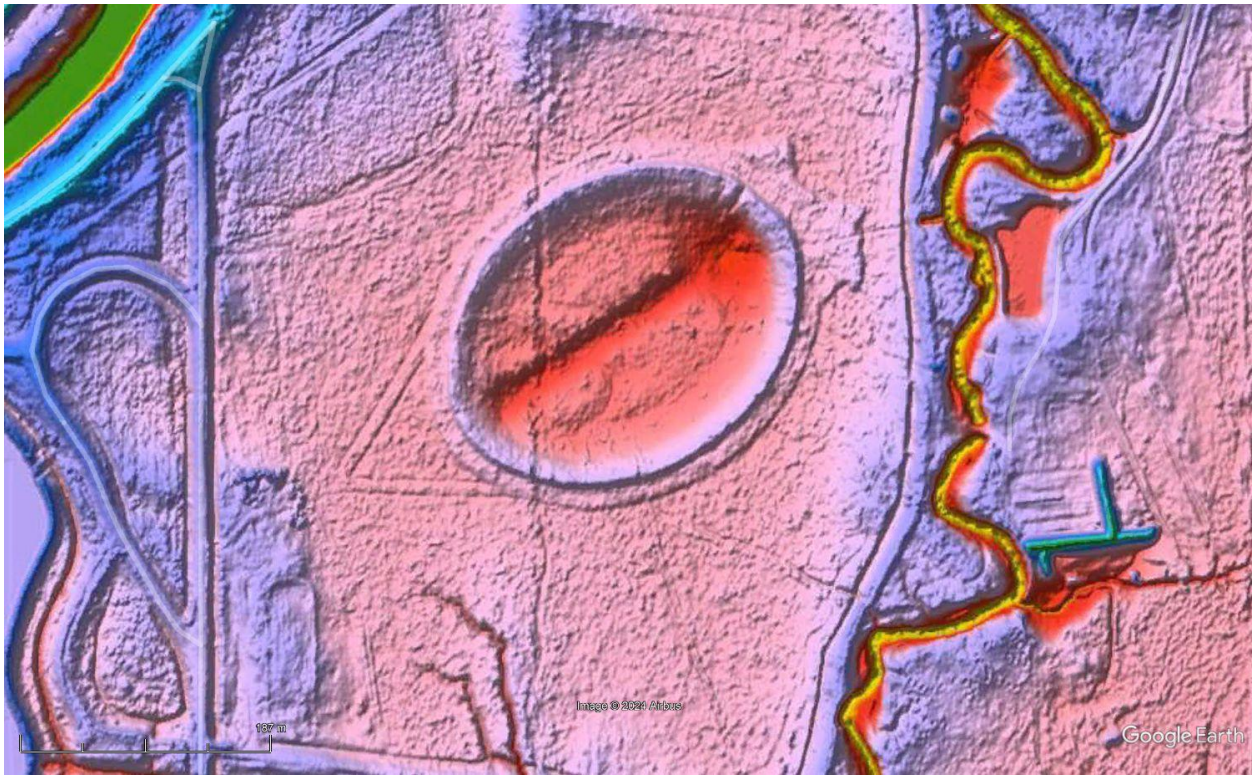
The Carolina Bays are shallow elliptical depressions on unconsolidated soil that originated as penetration funnels from secondary impacts of glacier ice boulders ejected by an extraterrestrial impact on the Laurentide Ice Sheet (Zamora, 2017; Zamora, 2020). Since the Carolina Bays are conic sections, it is often necessary to fit them with ellipses.



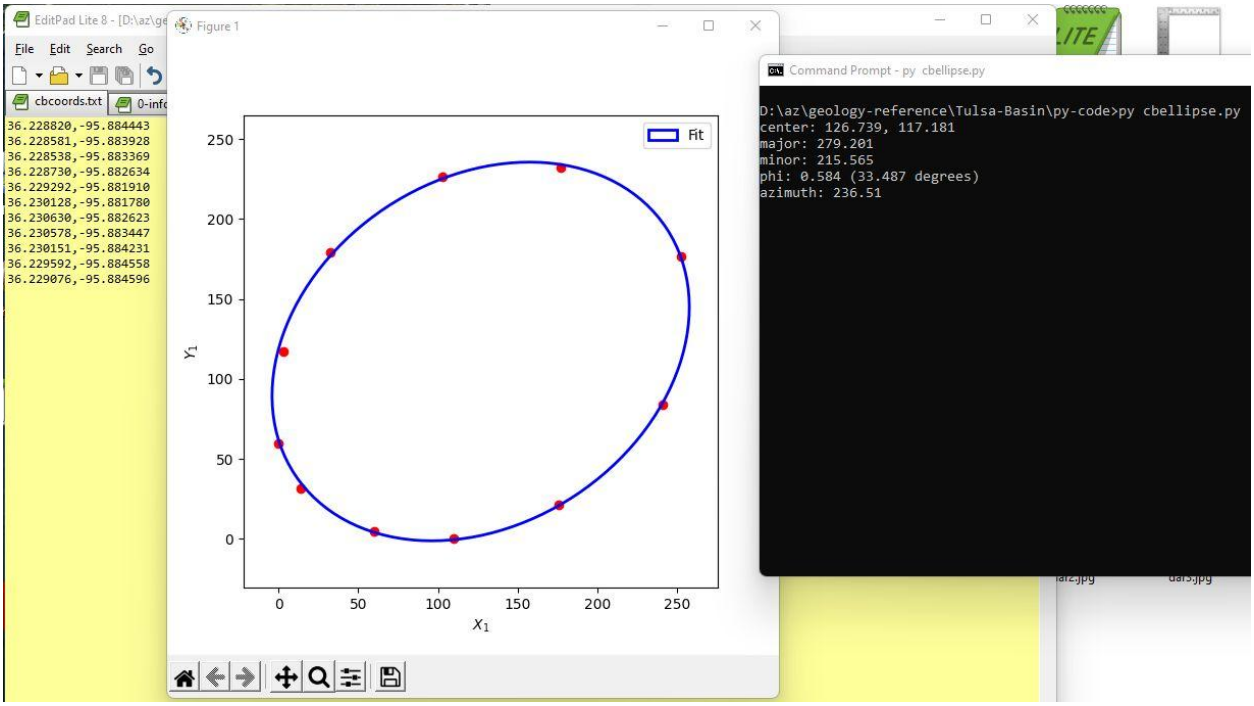
LiDAR image of Carolina Bays near Bowmore, NC
Three bays have been fitted with ellipses.

The best way to fit ellipses to the Carolina Bays is by identifying points along the perimeter of a bay and using a least squares method to fit the points to an elliptical curve. The basis for this program is a Python routine for least squares fitting of an ellipse based on a publication of Halir and Flusser [3]. The program **ellipse.py** calculates the coefficients of the polynomial for the ellipse.

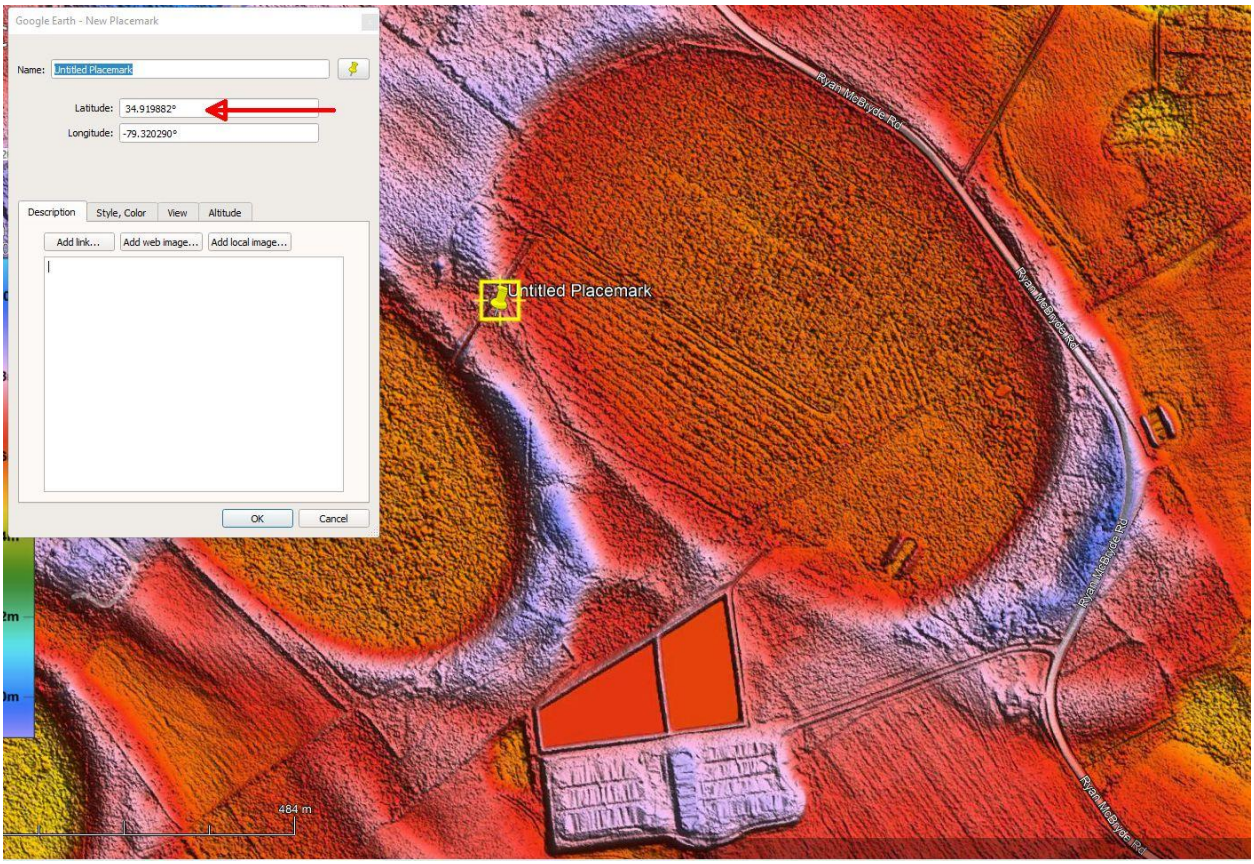
The program called **CBellipse.py** [5] reads a file of comma-separated latitude/longitude pairs and the points are changed to meters relative to the southmost and westmost positions so that the ellipse can be displayed in the first quadrant of a graph. The procedure for converting the coordinates to meters assumes that one degree of latitude is equal to 10,000,000 meters/90 degrees or 111,111 meters/degree. The distance in meters between degrees of longitude depends on the latitude. The program uses the cosine of the minimum latitude in the coordinate pairs and multiplies by 111,111.



LiDAR image of the Tulsa Basin. The coordinates of the rim are given in the next image.

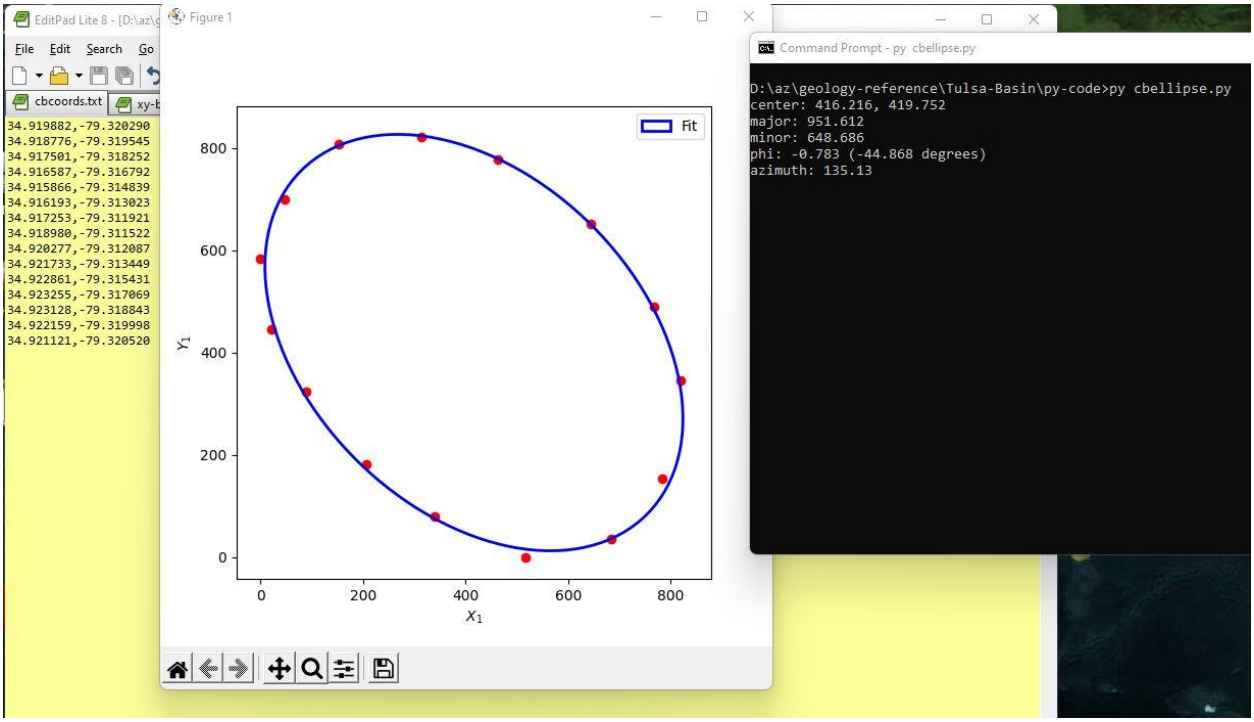


Results of processing the coordinate pairs of the Tulsa Basin



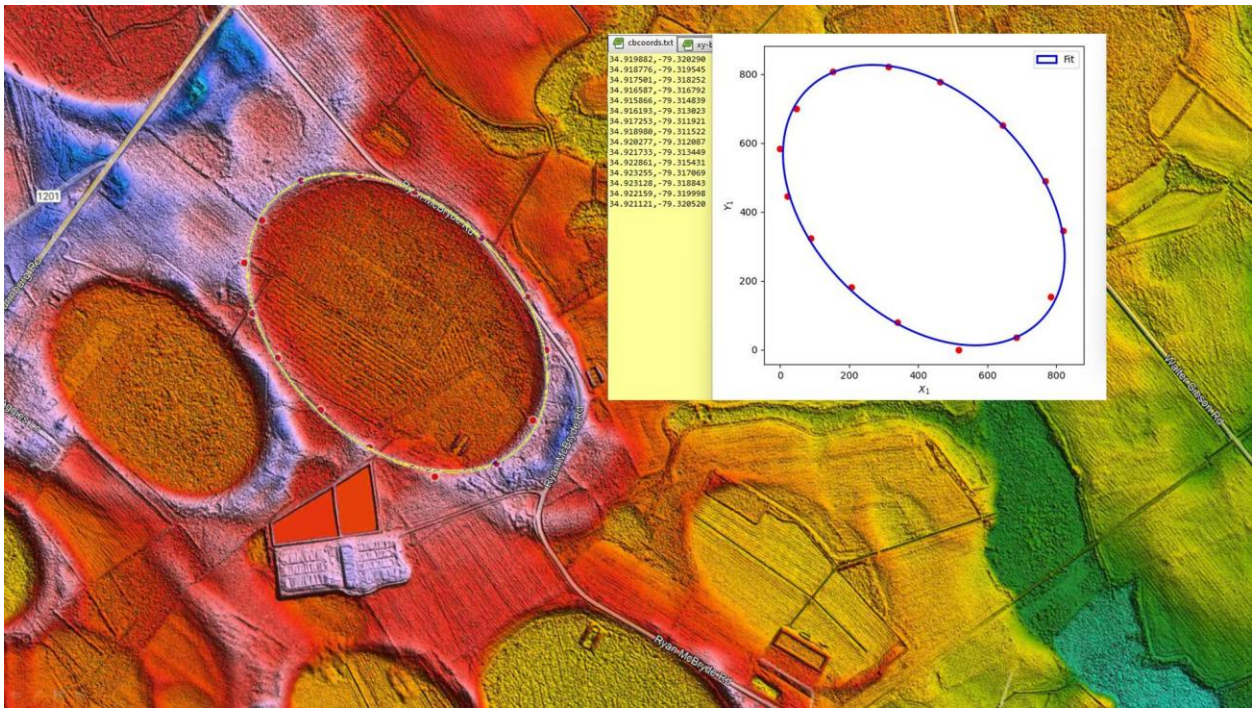
LiDAR image of Carolina Bays near Bowmore, NC.

The LiDAR visualization tool by Michael Davias [4] integrates with Google Earth and makes it possible to position the crosshairs of a pushpin along the rim of the bay in order to display a panel from which the coordinates for the latitude and longitude may be copied.



Results of processing the coordinate pairs of a Carolina Bay near Bowmore, NC

A second program called **driver2d.py** performs the functions of **CBellipse.py**, but it adds the capability of entering x,y coordinates from a digitized image instead of latitude and longitude by specifying *2D as a comment at the beginning of the data file.



The ellipse may be overlaid on the original image using PowerPoint or an image editor.

February 3, 2025: **driver2d.py** was updated to calculate the fitting error. This is done by obtaining the average error distance, which is the sum of the distance between the observed and calculated points, divided by the number of points. This makes the average error distance independent of the number of sample points. The fitting error is expressed as a percentage of the average error distance divided by the semiminor axis of the ellipse in order to scale the errors to the size of the ellipse. This makes it possible to compare fitting errors for ellipses of different sizes.

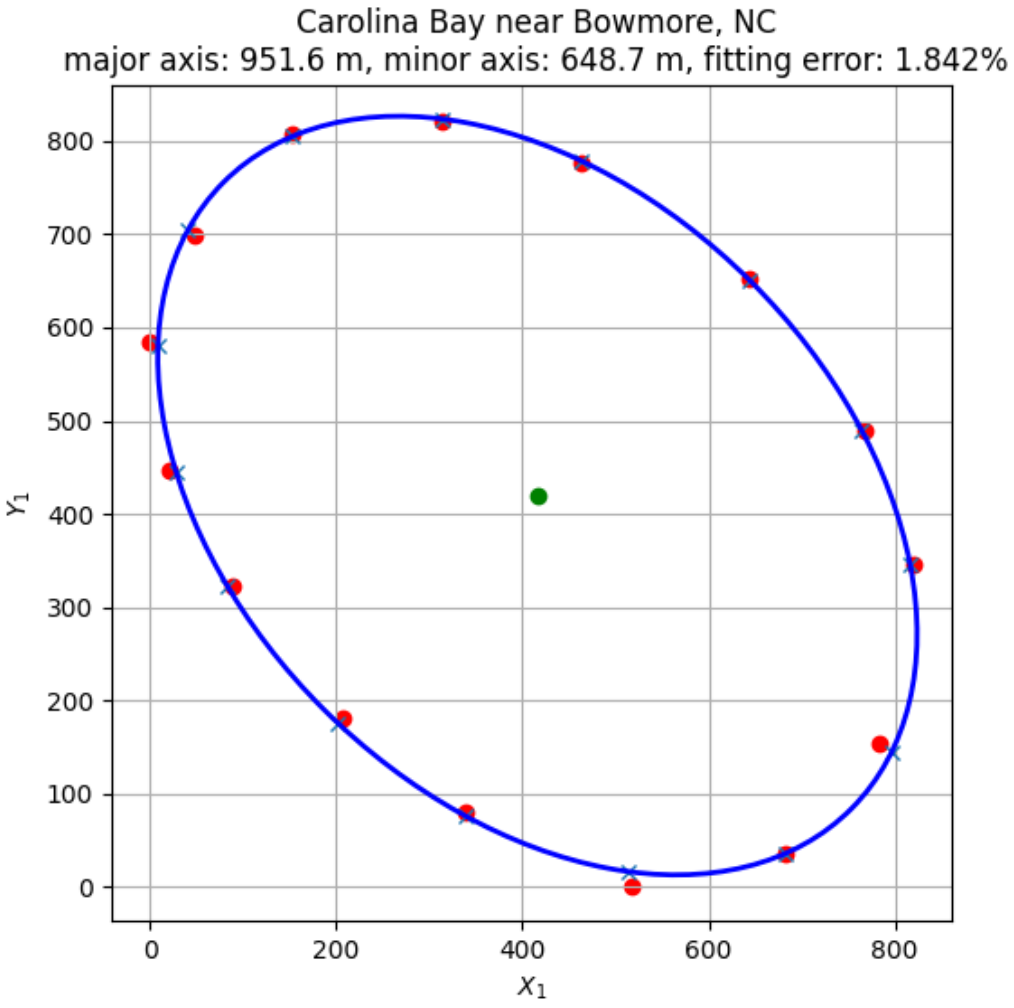
Example of input showing the option to display **grid** lines on the image. The option **ddt** can be added for debugging.

```
* Bay ID 139317-6726 (Bowmore)
*T= Carolina Bay near Bowmore, NC
*O= grid
34.919882,-79.320290
34.918776,-79.319545
34.917501,-79.318252
34.916587,-79.316792
34.915866,-79.314839
34.916193,-79.313023
34.917253,-79.311921
34.918980,-79.311522
34.920277,-79.312087
34.921733,-79.313449
34.922861,-79.315431
34.923255,-79.317069
34.923128,-79.318843
34.922159,-79.319998
34.921121,-79.320520
```

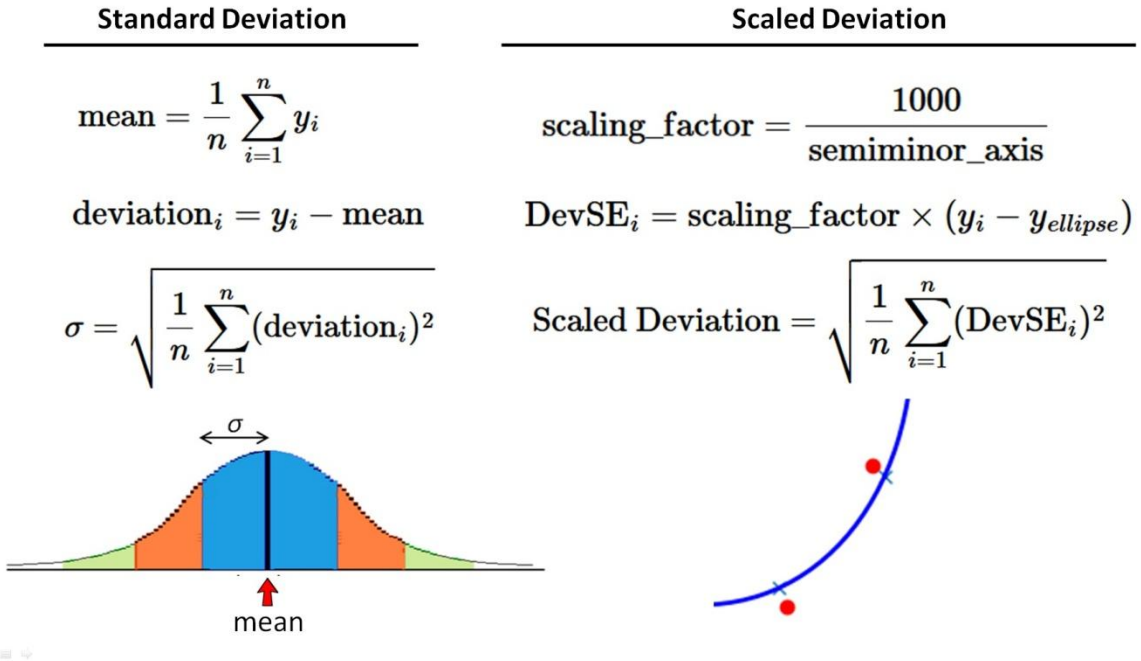
Results:

```
coefficients for  F(x,y) = ax**2 + 2bxy + cy**2 + 2dx + 2fy + g
a=-0.627, b=-0.459, c=-0.629, d=714.752, f=719.343, g=-209485.177
 Carolina Bay near Bowmore, NC
center: 416.216, 419.752
semimajor=475.8060, semiminor=324.3429, phi: 2.3585 (135.132 deg.)
scaling factor = 3.0831565043962383
Number of points = 15
Residual Sum of Squares (RSS) = 460.0128
Mean Squared Error (MSE) = 30.6675
Standard Deviation = 5.5378
```

Residual Sum of Scaled Squares (RSSS) = 4,372.8150
Scaled Mean Squared Error (SMSE) = 291.5210
Scaled Deviation = 17.0740
Mean Absolute Deviation (MAD) = 3.5829
MAD/semiminor = 1.1047%
Average error distance = 5.9735
Fitting error = 1.8417%
center Lat.,Lon. 34.919644, -79.315952
major: 951.612 m
minor: 648.686 m
area: 484,824.2 square meters
phi: 2.359 (135.132 degrees)
azimuth: 134.87



January 6, 2026: **driver2d.py** was updated to calculate the Scaled Mean Squared Error (SMSE) and the Scaled Deviation. The standard deviation is calculated based on the difference of the sample points from the mean, whereas the scaled deviation is calculated based on the difference between the sample points and the corresponding coordinates of the points on the ellipse, multiplied by the scaling factor. DevSE is the deviation corresponding to an ellipse with a semiminor axis of 1000. The following image illustrates the difference between the standard deviation and the scaled deviation.



Note on accuracy:

The conversion of geographical coordinates to meters assumes that there are 111,111 meters/degree and that the Earth is a sphere. However, the Earth is an oblate spheroid. The distance represented by 1° of latitude varies slightly with latitude, and longitude varies strongly with latitude. The current implementation introduces a latitude error of about 9 meters per degree and a longitude error of 1 to 10 meters depending on latitude and span. For the small distances involving Carolina Bays with a length of one kilometer, the overall expected error is less than the error in selecting a point along the perimeter of a bay, which depends on the resolution of the LiDAR image and the mechanism of selecting the point with Google Earth. The World Geodetic System WGS-84 ellipsoidal formulas can reduce the conversion error from several meters to centimeter-level for small spans using the following revised Python code:

```
import numpy as np
# --- Ellipsoidal formulas for meters per degree (WGS-84) ---
# Latitude-dependent length of one degree of latitude:
```

```
#   M = 111132.92 - 559.82*cos(2φ) + 1.175*cos(4φ)
#
# Longitude-dependent length of one degree of longitude:
#   N = 111412.84*cos(φ) - 93.5*cos(3φ)

def meters_per_degree_lat(phi_deg):
    phi = np.radians(phi_deg)
    return (111132.92
            - 559.82 * np.cos(2 * phi)
            + 1.175 * np.cos(4 * phi))

def meters_per_degree_lon(phi_deg):
    phi = np.radians(phi_deg)
    return (111412.84 * np.cos(phi)
            - 93.5 * np.cos(3 * phi))

# ------------------------------------------------------------
# Find minimum X and Y coordinates
if sw2d == 0:  # Lat/Lon coordinates were input
    minlat = min(list1)
    maxlat = max(list1)
    maxlon = max(list2)
    minlon = min(list2)

    if float(minlat) > 28.0 and float(maxlat) < 49.0 and \
       float(maxlon) > -105. and float(minlon) < -66.0:
        calc_azimuth = 1

else:  # plot coordinates were input
    minlat = min(list1, key=lambda x: float(x))
    maxlat = max(list1, key=lambda x: float(x))
    maxlon = max(list2, key=lambda x: float(x))
    minlon = min(list2, key=lambda x: float(x))

# ------------------------------------------------------------
# Compute meters-per-degree at the reference latitude
lat0 = float(minlat)

M_lat = meters_per_degree_lat(lat0)   # meters per degree
latitude
N_lon = meters_per_degree_lon(lat0)   # meters per degree
longitude

# ------------------------------------------------------------
# Convert latitude to meters
ellipse_y = []
for j in list1:
```

```
    if sw2d == 0:
        ellipse_y.append((float(j) - float(minlat)) * M_lat)
    else:
        ellipse_y.append(float(j) - float(minlat))

# Convert longitude to meters
ellipse_x = []
for j in list2:
    if sw2d == 0:
        dx_deg = abs(float(maxlon)) - abs(float(j))
        ellipse_x.append(dx_deg * N_lon)
    else:
        ellipse_x.append(float(j) - float(minlon))
```

References:

A. Zamora, 2017, A model for the geomorphology of the Carolina Bays, *Geomorphology*, 282, 209–216. (2017), DOI 10.1016/j.geomorph.2017.01.019 https://doi.org/10.1016/j.geomorph.2017.01.019

A. Zamora, Reply to Holliday et al. regarding the Carolina Bays, *Earth-Science Reviews*, Volume 261, 2025, 105024, ISSN 0012-8252, https://doi.org/10.1016/j.earscirev.2024.105024

A. Zamora, 2020. The Neglected Carolina Bays: Ubiquitous Geological Evidence of a Cataclysm, Paperback Edition (ISBN: 979-8677035999, 2020). https://amzn.to/4dqrcRY

Least Squares fitting of ellipses, python routine based on the publication Halir, R., Flusser, J.: 'Numerically Stable Direct Least Squares Fitting of Ellipses' https://github.com/bdhammel/least-squares-ellipse-fitting/releases/tag/v2.0.0

M. Davias, 2023, LiDAR High resolution Topology Model (HRTM) map of the 48 contiguous United States. http://lidar-hrtm.cintos.org/

A. Zamora, 2022. Python program for fitting ellipses to the Carolina Bays by the least squares method. https://github.com/citpeks/Carolina-Bays-least-squares-ellipse-fitting